

Capstone Project - Financial News Sentiment Analysis

James Cook University Cairns

Hunter Kruger-Ilingworth (14198489)

April 11, 2025

Contents

1	Abstract	2
2	Introduction	2
3	Web Crawler	3
3.1	Data Type Selection	3
3.2	Website Selection	3
3.3	Web Scraping	5
4	Data Wrangling and Feature Extraction	8
4.1	Exploratory Data Analysis	8
4.2	Sentiment Analysis	9
5	Machine Learning	11
6	References	12

List of Figures

1	Proportion of Each Stock Price Prediction method [1]	2
2	Proportion of Each of the Studies' Data Source [1]	2
3	Example Article [2], containing fields such as title, author, date, and stock codes	4
4	Quantity of Scraped Articles per Month	8
5	Distribution of Document Length	8
6	Distribution of Articles by Sector, using the <i>smallcaps</i> industry label and the ASX sector label [3]	8
7	Frequency of the top words in the scraped articles	9
8	Principle Component Analysis Biplot of the TF-IDF vectorised article corpus (with lemmatisation applied)	9
9	Distribution of Sentiment Labels and scores for VADER and BERT	10
10	Heatmap Comparison of sentiment scores of VADER vs BERT	10
11	Typical "FeedForward" Neural Network Topology [4]	11
12	Recurrent Neural Network (RNN) Topology [5]	11

List of Tables

1	Comparison of financial and market-related websites for article scraping suitability. Green rows indicate favourable conditions for scraping.	4
2	Number of Observations per Year As reported by the XML Sitemap Last Modified Date	5
3	Parsed dictionary example converted into a table	6

Abstract

This report investigates the feasibility of stock price prediction using NLP. Presented is an overview of the issue, a comparison of various textual sources, a literature review of the subject, and the detailing of development of a machine learning model to predict stock price movements, integrating sentiments and features extracted from smallcaps articles.

Introduction

need a proper introduction to the topic and THEN relating it to a data science problem

Predicting the movements of the stock market is a long standing challenge. Its erratic and volatile nature leads many to believe that predicting it is a fools game, with even experts in the field unable to consistently outperform the market [6–8]. The Efficient Market Hypothesis (EMH) claims that asset prices quickly reflect all available information [6]; it is also believed herd behavior and emotions can drive price swings beyond fundamentals [9]. It therefore follows that incorporating information and sentiment from sources across the internet may improve the performance of stock price prediction models to address this age old problem.

Many efforts have been made to predict stock price movements, with a variety of methods employed, with machine learning being particularly popular in recent years [1]. A literature review conducted by [1] analysed 81 papers from January 2015 to June 2020, finding that support vector machines (SVM) were the most popular method, with 26.13% of the analysed papers using them, followed by random forest (RF) (15.32%), and long short term memory (LSTM) (13.51%).

Sentiment, particularly in recent times on the world wide web present a unique opportunity to leverage in this space. Online financial discourse is heavily concentrated across a few key domains, including social platforms (e.g., Reddit's r/WallStreetBets), news aggregators (e.g., Yahoo Finance), and article-based financial outlets (e.g., Bloomberg), and market announcements [10]. These domains offer continuous textual data that offer differing levels of up to date information and consumer sentiment. fig. 2 shows that a vast majority of the studies analysed in the literature review by [1] used only technical indicators, and not taking advantage of news data - showing a potential opportunity to explore this field **unsure about this sentence**

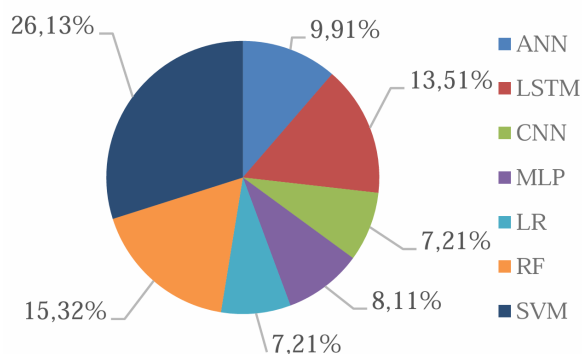


Fig. 1. Proportion of Each Stock Price Prediction method [1]

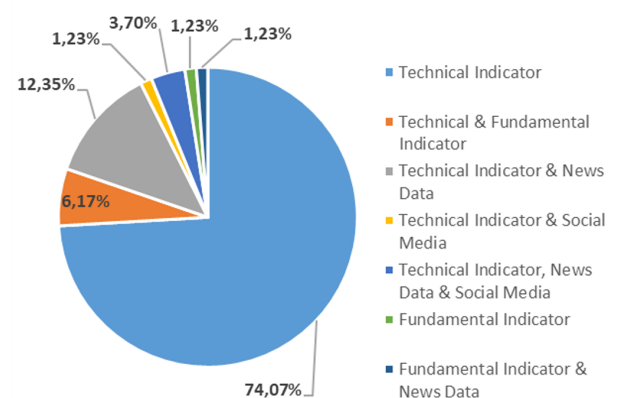


Fig. 2. Proportion of Each of the Studies' Data Source [1]

NLP techniques, not only have been proven applicable to the prediction of stock price, but can also offer value through reducing the burden of manual news monitoring and analysis that is required by a day trader.

This report presents a pipeline that collects news articles using web scraping techniques and processes them through NLP systems to extract meaningful insights. All code and reporting files are available on GitHub.

Web Crawler

Data Type Selection

Several data sources were considered for this investigation.

The ASX market announcements [10] were initially examined. These are direct from the source and released frequently, but are often mundane or irrelevant. They are also provided in PDF format, which is not conducive to text mining using tools such as `BeautifulSoup`. Furthermore, usage is restricted under the ASX's terms of service [11].

Social media platforms, particularly Twitter and Reddit, were also considered. These offer a large volume of data, albeit with short, unstructured content. Posts commonly include colloquial language, spelling errors, requiring extensive preprocessing [12]; furthermore being less factual and trustworthy than other sources [13]. Unlike financial journalists, social media users are not held to standards of factual accuracy, limiting the reliability of this source.

In contrast, financial news articles are typically factual, well-written, and longer in length. Web-based sources often provide a more structured format and accessible metadata, making them better suited to text mining. It is for these reasons that financial news articles were selected as the primary data source for this investigation.

Website Selection

Table 1 shows the comparison of various websites that contain financial and market-related articles, evaluating their suitability for scraping. Copyright is a key legal consideration when using original written works. Under Section 113P of the Copyright Act 1968, Australian copyright law permits educational institutions to copy and communicate works for educational purposes under the 'fair dealing' provisions [14].

Websites were assessed based on their suitability for article scraping, with key criteria including explicit permission in terms of use, presence of disallow/allow directives in `robots.txt`, use of scraping inhibiting in the form of Cloudflare [15], and article structure (e.g. presence of stock codes or site metadata). As shown in Table 1, many domains impose legal or technical restrictions on automated data collection, presenting ethical limitations to their use. For example, scraping from websites that explicitly disallow it in their terms of service may breach legal and institutional ethical standards. From a sampling design perspective, sites with structured article metadata and consistent tagging (e.g. Trading Economics, Stock Head) were preferred to ensure replicable and unbiased data collection. Conversely, general news outlets with mixed content and unstructured articles (e.g. ABC News, Yahoo Finance) were excluded due to increased noise and ambiguity in topic association.

Small Caps was selected due to its static structure, enabling lightweight scraping using `BeautifulSoup`. All content is stock-focused, removing the need for filtering, and articles are consistently tagged with stock codes, simplifying parsing. However, using a single source limits dataset diversity and may bias coverage towards small-cap companies, or a particular sector, or reflect the opinions of a few authors.

Website Name (URL hyperlinked)	Scrapping Permitted in Terms of Use?	Allows Article Scrapping in Robots.txt?	Uses Cloudflare?	General Notes
Market Index	No	Yes	No	Company tags on articles
Yahoo Finance	No	No	No	no mentioned stock labelling. General politics also included which is difficult to relate to individual stocks
Trading Economics	Yes	Yes	No	General politics and economics also included — would require filtering (unstructured)
Financial Review	No	No	No	
The Motley Fool	No	Yes	No	
Sydney Morning Herald	No	No	No	
The Market Online	No	Yes	No	
ABC News	Yes	Yes	No	Regular news mixed with financial news, difficult to use, untagged stock market
Small Caps	Yes	Yes	No	Sitemap highly conducive to scraping. Stock codes provided in articles. All about stocks — no general politics. Static web content (use BeautifulSoup)
Morning Star	No	Yes	No	No sitemap provided
Australian Stock Report	Yes	Yes	No	Unstructured in comparison to other article sites
Stock Head	Yes	Yes	No	Great sitemap but no list of stock codes in article
The Australian	No	No	No	
Hot Copper	No	Yes	No	

Table 1: Comparison of financial and market-related websites for article scraping suitability. Green rows indicate favourable conditions for scraping.

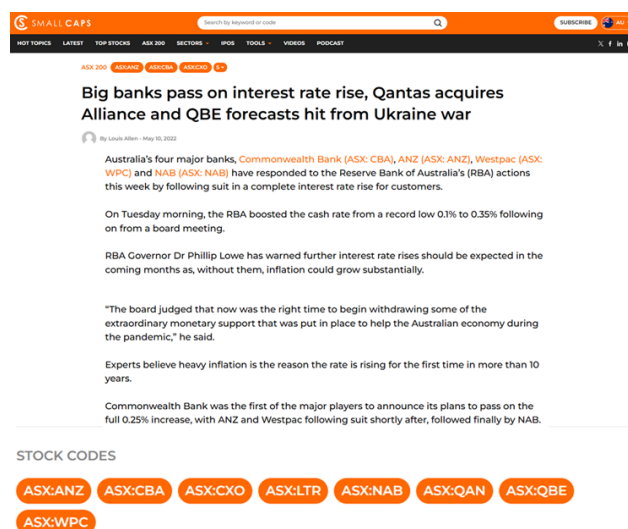


Fig. 3. Example Article [2], containing fields such as title, author, date, and stock codes

Web Scraping

Although `robots.txt` did not specify a scraping frequency, throttling was implemented to reduce server load as a matter of ethical practice. *Small Caps* was selected in part due to its exposed XML sitemap, which simplifies article discovery by listing URLs directly—avoiding the need to emulate navigation behaviour using heavier tools like *Selenium*. Instead, the sitemap is parsed using `lxml` with `BeautifulSoup`, which is significantly faster and less resource-intensive for static content.

Given the site's predictable structure and absence of JavaScript rendering, *Selenium* was deemed unnecessary. This reduces scraping complexity and improves runtime efficiency. listing 1 shows the sitemap scraping function, which extracts article metadata including title and last-modified date. As shown in listing 2, each paginated sitemap is queried sequentially, filtered for article URLs, and written to a local CSV file. This storage format was chosen for ease of inspection and downstream processing.

Table 2 shows the temporal distribution of articles scraped, based on the last modified date in the sitemap.

Listing 1: Definition of xml sitemap scraping function

```
1 import requests
2 from bs4 import BeautifulSoup
3 from datetime import datetime
4
5 def extract_article_metadata_list_from_sitemap(url,
6     sitemap_number):
7     """
8     Iterate through a sitemap and for each url tag, extracts
9     the URL, title, last modified timestamp, year, month,
10    and day.
11    Returns a list of dictionaries, each containing the
12    extracted metadata, for the given sitemap URL.
13    """
14    response = requests.get(url)
15    if response.status_code != 200:
16        print(f"Failed to fetch the sitemap. Status code: {
17            response.status_code}")
18        return []
19
20    article_observations = []
21    soup = BeautifulSoup(response.content, 'lxml-xml')
22    for url_tag in soup.find_all('url'):
23        loc_tag = url_tag.find('loc')
24        lastmod_tag = url_tag.find('lastmod')
25
26        if loc_tag is None:
27            continue # skip if <loc> is missing
28
29        loc_tag_text = loc_tag.text.strip()
30        last_modified = lastmod_tag.text.strip() if
31        lastmod_tag else None
32
33        # infer report title from url
34        title = loc_tag_text.replace("https://smallcaps.com.
35        au", "").replace("-", " ").replace("/", "").capitalize
36        ()
37
38        lastmod_timestamp = None
39        if last_modified:
40            try: # convert last_modified to a datetime
41                object
42                lastmod_timestamp = datetime.strptime(
43                last_modified, '%Y-%m-%dT%H:%M:%S%z').timestamp()
44            except Exception as e:
45                print(f"Error parsing date {last_modified}:
46                {e}")
47
48        article_observations.append({
49            "url": loc_tag_text,
50            "sitemap_number": sitemap_number,
51            "title": title,
52            "last_modified_timestamp": lastmod_timestamp,
53            "last_modified_year": last_modified[:4] if
54            last_modified else None, # Year format
55            "last_modified_month": last_modified[5:7] if
56            last_modified else None, # Month format
57            "last_modified_day": last_modified[8:10] if
58            last_modified else None # Day format
59        })
60    return article_observations
```

Listing 2: Code to use the xml scraping function

```
1 import pandas as pd
2
3 is_article_url = lambda url: len(url) >= 50 # article
4 titles are included in the url, so we can use this
5 distinction to filter out non-article URLs
6
7 sitemap_count = 66
8 articles = []
9 for sitemap_number in range(1, sitemap_count + 1):
10     print(f"Fetching sitemap {sitemap_number}")
11     try: # there are a list of sitemaps with the names part
12         /1.xml, part/2.xml, we gather all urls from all
13         sitemaps and append to articles list
14         sitemap_sub_path = f'sitemap/part/{sitemap_number}.
15         xml'
16         article_metadata =
17         extract_article_metadata_list_from_sitemap(f'https://
18         smallcaps.com.au/{sitemap_sub_path}', sitemap_number)
19         article_metadata = list(filter(lambda url_data:
20         is_article_url(url_data['url']), article_metadata))
21         articles.extend(article_metadata)
22     except Exception as e:
23         print(f"Failed to fetch sitemap {sitemap_number}.
24         Error: {e}")
25
26 df = pd.DataFrame(articles)
27 df.to_csv('../data/article_metadata.csv', index=False)
```

Year	Number of Observations
2023	10,468
2024	2,374
2025	323
Total	13,165

Table 2: Number of Observations per Year As reported by the XML Sitemap Last Modified Date

Once the sitemap URLs were gathered, article content was scraped individually; once again using the

Key	Value
'title'	'Big banks pass on int...'
'author'	'Louis Allen'
'date'	'May 10, 2022'
'sector'	'ASX200'
'stock_codes'	['ASX: CBA', 'ASX: ANZ',...]
'document'	"Australia's four maj..."

Table 3: Parsed dictionary example converted into a table

`requests` library and `BeautifulSoup` with the `lxml` parser. The article scraping function is shown in listing 3, where it is seen that the target data with the key metadata resided within HTML containers, enabling reliable extraction of fields such as article title, author, date, sector, body text, and stock codes.

To allow for incremental scraping and avoid reprocessing, existing scraped articles were tracked via CSV and excluded in subsequent runs. Requests were throttled with a 3-second delay between each fetch to reduce server load. Output was stored locally in a raw CSV format, enabling later augmentation and processing while keeping the collection process transparent and reproducible. The scraping code can be seen in listing 4.

Listing 3: Article Scraping function

```

1 import re
2 import urllib.request
3 from bs4 import BeautifulSoup
4
5 def extract_html_from_url(url):
6     """
7     given a url, return a soup object which contains the html content of the page
8     """
9     request = urllib.request.Request(url, headers={'User-Agent': 'Mozilla/5.0'})
10    html_content = urllib.request.urlopen(request).read().decode('utf-8')
11    soup = BeautifulSoup(html_content, 'lxml')
12    return soup
13
14 def parse_html_to_dictionary(soup_obj):
15     """
16     Parse the HTML content of a Small Caps article into a dictionary, extracting the title, author, date, sector, stock codes,
17     and document.
18     """
19    title_tag = soup_obj.find("h1", class_="c-post-header__title")
20    if title_tag: # Parse the title from the <h1> element;
21        title = title_tag.get_text(strip=True)
22    else: # fall back to <title> if not found.
23        title = soup_obj.title.get_text(strip=True) if soup_obj.title else None
24
25    # parse the author from the header
26    author_tag = soup_obj.find('a', class_='c-post-header__author-link')
27    author = author_tag.get_text(strip=True) if author_tag else None
28
29    # parse the date from the first <time> tag
30    date_tag = soup_obj.find('time')
31    date_string = date_tag.get_text(strip=True) if date_tag else None
32
33    # parse the sector from the navigation
34    sector = None
35    nav_section = soup_obj.find("div", class_="c-post__navigation")
36    if nav_section:
37        sector_tag = nav_section.find("a", class_="c-post__navigation-link")
38        sector = sector_tag.get_text(strip=True) if sector_tag else None
39
40    # parse the stock codes
41    stock_codes = []
42    # look for any div whose class attribute contains "c-tags"
43    tags_container = soup_obj.find(
44        lambda tag: tag.name == "div" and tag.get("class") and any("c-tags" in c for c in tag.get("class"))
45    )
46    if tags_container:
47        stock_text_span = tags_container.find("span", class_="c-tags__text")
48        if stock_text_span and "Stock Codes" in stock_text_span.get_text():
49            # Try to find an inner container if available
50            inner = tags_container.find("div", class_="c-tags__inner")
51            if inner:
52                stock_codes = [a.get_text(strip=True) for a in inner.find_all("a", class_="c-tags__item c-tags__item--link")]
53            else:
54                stock_codes = [a.get_text(strip=True) for a in tags_container.find_all("a", class_="c-tags__item c-tags__item

```

```

--link"]])
55
56 # parse the article body
57 body_tag = soup_obj.find("div", class_="c-rich-text c-post__rich-text")
58 if body_tag:
59     # Use "\n" as separator to preserve paragraph breaks
60     document = body_tag.get_text(separator = " ", strip=True)
61     document = re.sub(r'[\w\s]', '', document) if document else None # use regex to remove any character that is not a
    word character or whitespace
62 else:
63     document = None
64
65 return {
66     'title': title,
67     'author': author,
68     'date_string': date_string,
69     'sector': sector,
70     'stock_codes': stock_codes,
71     'document': document
72 }

```

Listing 4: Code to Parse Websites in xml Sitemap

```

1 import pandas as pd
2
3 is_article_url = lambda url: len(url) >= 50 # article titles are included in the url, so we can use this distinction to
    filter out non-article URLs
4 sitemap_count = 66
5 articles = []
6 for sitemap_number in range(1, sitemap_count + 1):
7     print(f"Fetching sitemap {sitemap_number}")
8     try: # there are a list of sitemaps with the names part/1.xml, part/2.xml, we gather all urls from all sitemaps and
        append to articles list
9         sitemap_sub_path = f'sitemap/part/{sitemap_number}.xml'
10        article_metadata = extract_article_metadata_list_from_sitemap(f'https://smallcaps.com.au/{sitemap_sub_path}',
            sitemap_number)
11        article_metadata = list(filter(lambda url_data: is_article_url(url_data['url']), article_metadata))
12        articles.extend(article_metadata)
13    except Exception as e:
14        print(f"Failed to fetch sitemap {sitemap_number}. Error: {e}")
15 df = pd.DataFrame(articles)
16 df.to_csv('../data/article_metadata.csv', index=False)

```

The contents of the HTML were analysed and parsed into a dictionary capturing the title, author, date, sector, stock codes, and the article body, as demonstrated in the listing below. **BeautifulSoup** was used to extract data from the relevant tags. Below is an example output from the parsing process:

Data Wrangling and Feature Extraction

Exploratory Data Analysis

Sample Distribution

Table 3 shows the structure of the dataset. A total of 1962 articles were collected from Smallcaps. Figure 4 shows the time distribution over time, covering all of 2024, late 2023, and early 2025. This time window may be a limitation, as older articles could exhibit different characteristics not captured in the current dataset. Figure 5 depicts the document length distribution, with a mean word count of 462. The dataset size and document length are sufficient for training machine learning models, particularly when using pretrained transformers such as BERT [16] or FinBERT [17].

Listing 7 shows the addition of industry labels to articles using a list of stock codes retrieved from the ASX website [3]. These labels, along with those reported by Smallcaps, are shown in fig. 6. Most articles fall within the mining sector, indicating a sample bias. While ASX labels offer greater context, the larger amount of labels introduces sparsity, with several sectors having very few observations.

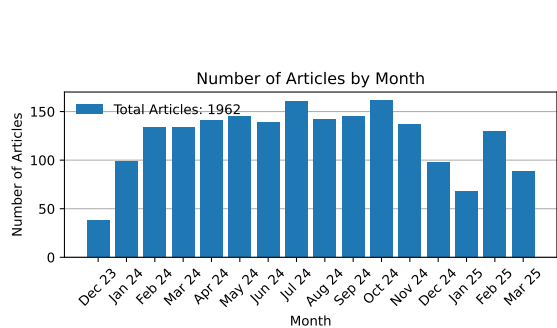


Fig. 4. Quantity of Scraped Articles per Month

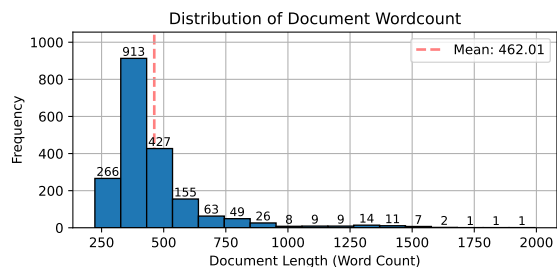


Fig. 5. Distribution of Document Length

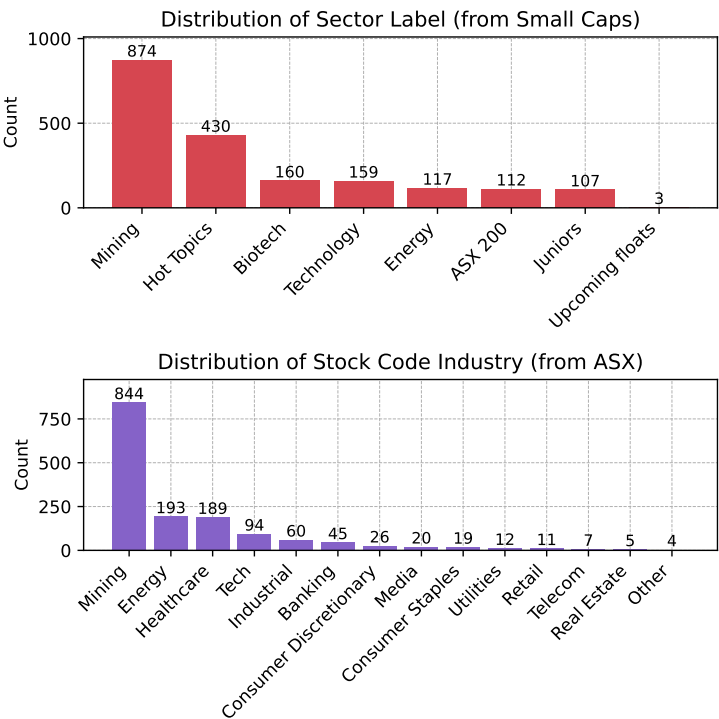


Fig. 6. Distribution of Articles by Sector, using the *smallcaps* industry label and the ASX sector label [3]

Text Corpus

Figure 7 shows the most common words in the articles, whilst removing stop words using `Sklearn CountVectorizer`. It can be seen that past general terms like company, project and asx, mining and resource terms are the most prevalent in the corpus, further demonstrating the mining sampling bias highlighted in fig. 6

Figure 8 shows the PCA biplot of the TF-IDF vectorised article corpus, generated using `SKlearn`'s `PCA` and `TfidfVectorizer` modules. Lemmatisation was applied via `nltk` to reduce term dimensionality. It can be seen that even with two components, the plot shows some level of clustering by sector, suggesting that term usage differs meaningfully between them.

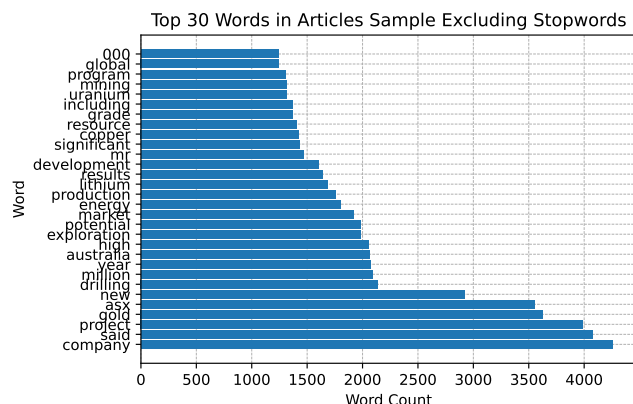


Fig. 7. Frequency of the top words in the scraped articles

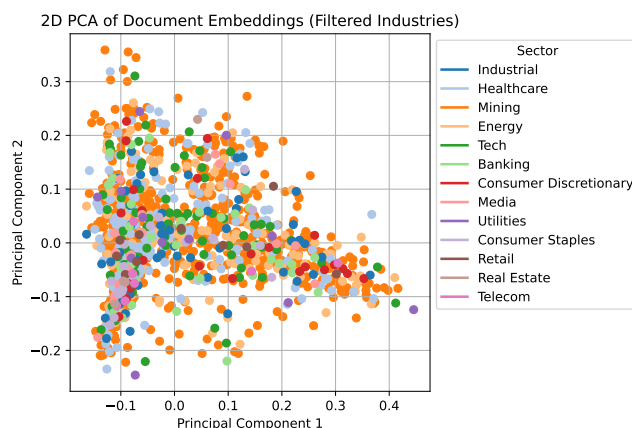


Fig. 8. Principle Component Analysis Biplot of the TF-IDF vectorised article corpus (with lemmatisation applied)

Sentiment Analysis

We model the sentiment of an article using the VADER (Valence Aware Dictionary and sEntiment Reasoner)[18], as this has shown to having promising results in a similar setting, as found by [13]. Implementation of VADER can be seen in listing 5, with the implementation inspired assisted by online article [19]

Listing 5: VADER sentiment analysis of article

```
1 from nltk.stem import PorterStemmer
2 from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer # apply lemmization first
3
4 analyzer = SentimentIntensityAnalyzer()
5 def get_vader_sentiment_scores(document, lemmize_document=False):
6     """Clean the text, and calculate VADER sentiment scores"""
7     if lemmize_document:
8         document = [PorterStemmer().stem(word) for word in document.split()]
9     return analyzer.polarity_scores(document)
10
11 sentiment_scores = articles_df['document'].apply(get_vader_sentiment_scores)
12
13 sentiment_scores_df = sentiment_scores.apply(pd.Series).rename(
14     columns={'pos': 'vader_positive', 'neg': 'vader_negative', 'neu': 'vader_neutral', 'compound': 'vader_compound'})
15 )
16 # add the new features to articles_df
17 articles_df = pd.concat([articles_df, sentiment_scores_df], axis=1)
```

VADER's rule-based approach relies on fixed lexicons and doesn't fully capture the context-dependent sentiment embedded in text, whereas BERT's deep contextual embeddings and bidirectional architecture captures nuanced meanings in multi-word expressions, and been pre-trained on large-scale text, including Wikipedia, BERT can recognise **swap up the wording here a bit** proper nouns and entities, making it effective for disambiguating stock names, tickers, and countries [16]. FinBERT is a domain-specific adaptation of the BERT model, fine-tuned on financial texts to accurately determine the sentiment expressed in this particular context [20]. This model was retrieved using the huggingface library [17].

While this sounds better, it relies on the data being labelled, making training on a large corpus of data unfeasible without labourious labelling. We therefore utilise the finbert-tone pretrained transformer which is the FinBERT model trained on 10,000 manually annotated documents (positive, negative, neutral) sentences from analyst reports [21].

Listing 6: BERT sentiment analysis of articles

```
1 from transformers import BertTokenizer, BertForSequenceClassification
2 from transformers import pipeline
3
4 finbert = BertForSequenceClassification.from_pretrained('yiyanghkust/finbert-tone', num_labels=3)
5 tokenizer = BertTokenizer.from_pretrained('yiyanghkust/finbert-tone')
```

```

6 nlp = pipeline("sentiment-analysis", model=finbert, tokenizer=tokenizer)
7
8 def get_bert_sentiment(doc):
9     truncated_text = tokenizer.decode(tokenizer.encode(doc, max_length=510, truncation=True))
10    result = nlp(truncated_text)[0] # Call nlp only once
11    return pd.Series({'bert_sentiment_label': result['label'], 'bert_sentiment_score': result['score']})
12
13 articles_df[['bert_sentiment_label', 'bert_sentiment_score']] = articles_df['document'].apply(get_bert_sentiment)
14 articles_df

```

It is evident from fig. 9 that VADER and BERT has a high skew towards positive sentiment, potentially indicating that the authors of smallcaps are averse to publishing negative news. Figure 10 shows that other than a small amount of outliers, the BERT and VADER sentiment models are highly correlated.

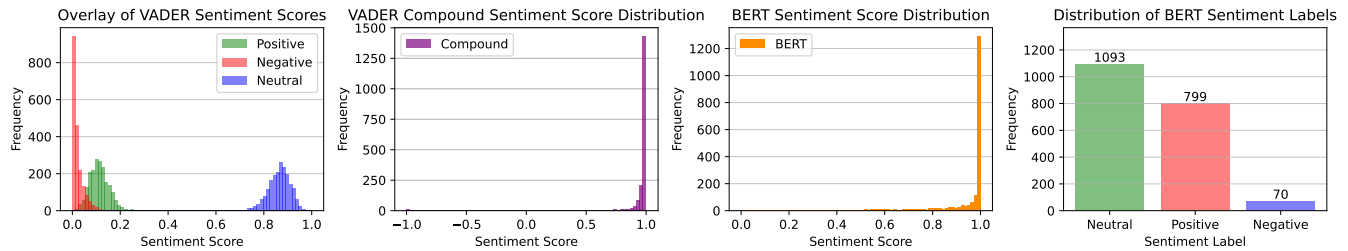


Fig. 9. Distribution of Sentiment Labels and scores for VADER and BERT

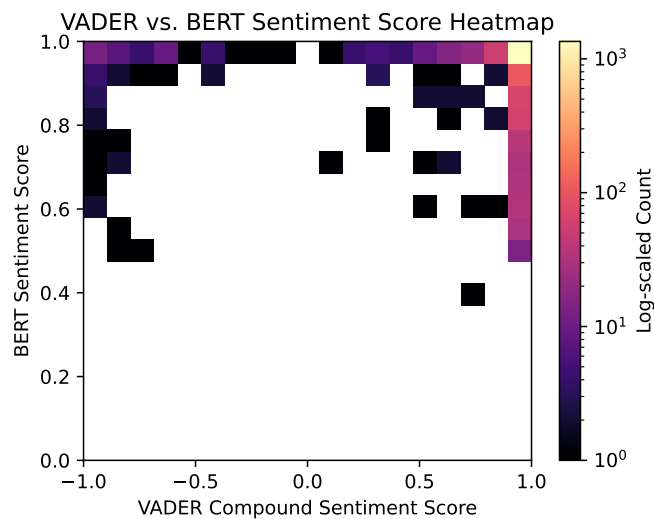


Fig. 10. Heatmap Comparison of sentiment scores of VADER vs BERT

Machine Learning

After having scraped the articles for the documents and sentiments? a machine learning model was developed to predict stock price. Machine learning, and particularly neural networks, are prevalent in the field of stock price prediction [1]. Figures 11 and 12 show the structure of a typical feed forward neural network and a recurrent neural network. Recurrent neural networks (RNNs), and others like RNNs, are designed to solve time series problems, and are therefore particularly well suited to this task. RNNs, while powerful, have difficulties with 'memory' due to the number of layers after multiple recursions; so a subset of RNNs, LSTMs allow retaining information for longer periods of time [22].

[22] show that an LSTM model, when trained on timeseries price data and sentiment data from news articles, can outperform a model trained on price data alone. Analysis of the literature suggests that SVM models, whilst the most popular [1] often underperformed in a financial setting, due to parameter optimisation being required [1,23]

Talk about the adam optimiser According to Kingma et al., 2014, the method is "computationally efficient, has little memory requirement, invariant to diagonal rescaling of gradients, and is well suited for problems that are large in terms of data/parameters". [24,25]

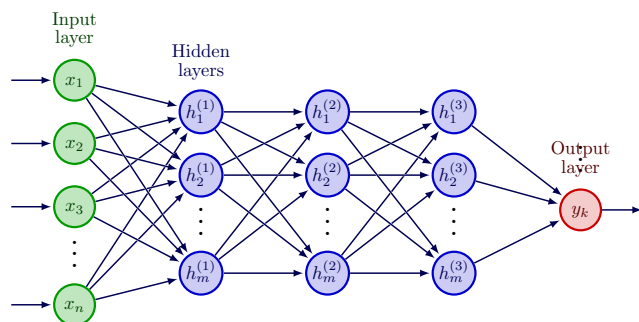


Fig. 11. Typical "FeedForward" Neural Network Topology [4]

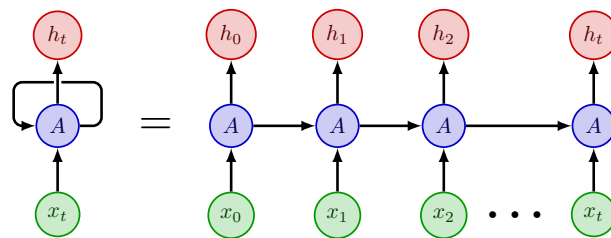


Fig. 12. Recurrent Neural Network (RNN) Topology [5]

the events extracted from the news are usually quite sparse, resulting in unreliable and unsustainable predictive power [23] (bar for bar) therefore we pivot and try and predict on the asx index which should address sparsity issues, rather than trying to predict volatile single stocks

References

- [1] A. Y. Al-Dulaimi, S. N. Al-Saadi, and S. A. Jasim, “The stock exchange prediction using machine learning techniques: A comprehensive and systematic literature review,” *Jurnal Ilmu Komputer dan Informasi*, vol. 14, no. 2, pp. 132–150, July 2021, accessed: 2025-04-07. [Online]. Available: <https://www.researchgate.net/publication/353002279>
- [2] L. Allen. (2022, May) Big banks pass on interest rate rise, qantas acquires alliance and qbe forecasts hit from ukraine war. <https://smallcaps.com.au/big-banks-interest-rate-rise-qantas-acquires-alliance-qbe-forecasts-hit/>. Accessed: 2025-04-02.
- [3] ASX Limited, “Company directory,” 2024, accessed: 2024-04-08. [Online]. Available: <https://www.asx.com.au/markets/trade-our-cash-market/directory>
- [4] IWN, “Drawing neural network with tikz,” <https://tex.stackexchange.com/questions/153957/drawing-neural-network-with-tikz>, October 2021, answer posted on TeX StackExchange, Accessed: 2025-04-07. [Online]. Available: <https://tex.stackexchange.com/questions/153957/drawing-neural-network-with-tikz>
- [5] user121799, “How do i draw a simple recurrent neural network with goodfellow’s style?” <https://tex.stackexchange.com/questions/494139/how-do-i-draw-a-simple-recurrent-neural-network-with-goodfellows-style>, June 2019, answer posted on TeX StackExchange, Accessed: 2025-04-07. [Online]. Available: <https://tex.stackexchange.com/questions/494139/how-do-i-draw-a-simple-recurrent-neural-network-with-goodfellows-style>
- [6] L. Downey. (2024) Efficient market hypothesis (emh): Definition and critique. Accessed: 2025-04-09. [Online]. Available: <https://www.investopedia.com/terms/e/efficientmarkethypothesis.asp>
- [7] L. Swedroe. (2019) The surprising results from s&p’s latest spiva analysis. Accessed: 2025-04-09. [Online]. Available: <https://www.advisorperspectives.com/articles/2019/11/18/the-surprising-results-from-s-ps-latest-spiva-analysis>
- [8] K. Gunnars. (2024) Is it really true that almost no one can beat the market? Accessed: 2025-04-09. [Online]. Available: <https://stockanalysis.com/article/can-you-beat-the-market/>
- [9] T. Greyling and S. Rossouw, “Twitter sentiment and stock market movements: The predictive power of social media,” <https://cepr.org/voxeu/columns/twitter-sentiment-and-stock-market-movements-predictive-power-social-media>, Mar. 2025, voxEU Column, Centre for Economic Policy Research.
- [10] ASX Limited, “Today’s announcements,” <https://www.asx.com.au/markets/trade-our-cash-market/todays-announcements>, 2025, accessed: 2025-04-02.
- [11] —, “Terms of use,” <https://www.asx.com.au/legals/terms-of-use>, 2023, last updated 12 October 2023. Accessed: 2025-04-02.
- [12] S. V. Kolasani and R. Assaf, “Predicting stock movement using sentiment analysis of twitter feed with neural networks,” *Journal of Data Analysis and Information Processing*, vol. 8, pp. 309–319, 2020.
- [13] A. Heiden and R. S. Parpinelli, “Applying lstm for stock price prediction with sentiment analysis,” in *Proceedings of the Brazilian Congress on Computational Intelligence (CBIC)*. Joinville, SC, Brazil: Santa Catarina State University (UDESC), January 2021.
- [14] “Copyright act 1968,” <https://www.legislation.gov.au/Series/C1968A00063>, 2024, in force. Administered by Attorney-General’s Department.

- [15] “Doesitusecloudflare.com,” <http://www.doesitusecloudflare.com>, 2025, accessed: 2025-04-09.
- [16] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2019, available at <https://doi.org/10.48550/arXiv.1810.04805>. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [17] ProsusAI, “Finbert [model],” n.d., retrieved March 27, 2025, from Hugging Face. [Online]. Available: <https://huggingface.co/ProsusAI/finbert>
- [18] C. J. Hutto and E. Gilbert, “VADER: A parsimonious rule-based model for sentiment analysis of social media text,” in *Proceedings of the Eighth International Conference on Weblogs and Social Media (ICWSM-14)*. AAAI Press, 2014. [Online]. Available: <https://www.researchgate.net/publication/275828927>
- [19] GeeksforGeeks, “Python sentiment analysis using VADER – using python,” December 2024, accessed: 2025-04-08. [Online]. Available: <https://www.geeksforgeeks.org/python-sentiment-analysis-using-vader/>
- [20] D. T. Araci, “Finbert: Financial sentiment analysis with pre-trained language models,” Master’s thesis, University of Amsterdam, 2019, preprint available on arXiv. [Online]. Available: <https://doi.org/10.48550/arXiv.1908.10063>
- [21] Y. Yang, “Finbert-tone: Fine-tuned financial sentiment model,” <https://huggingface.co/yiyanghkust/finbert-tone>, 2022, accessed: 2025-04-10.
- [22] C.-R. Ko and H.-T. Chang, “Lstm-based sentiment analysis for stock price forecast,” *PeerJ Computer Science*, vol. 7, no. 1, p. e408, March 2021.
- [23] X. Zhang, Y. Li, S. Wang, B. Fang, and P. S. Yu, “Enhancing stock market prediction with extended coupled hidden markov model over multi-sourced data,” *arXiv preprint arXiv:1809.00306*, September 2018, 19 pages. Submitted on 2 Sep 2018. Accessed: 2025-04-07. [Online]. Available: <https://doi.org/10.48550/arXiv.1809.00306>
- [24] Keras Team, “Adam class api docs,” <https://keras.io/api/optimizers/adam/>, accessed: 2025-04-07.
- [25] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” 2015, published as a conference paper at the 3rd International Conference for Learning Representations, San Diego, 2015. [Online]. Available: <https://doi.org/10.48550/arXiv.1412.6980>

Listing 7: Article Data Augmentation Code

```

1 asx_companies_df = pd.read_csv('../data/ASXListedCompanies.csv')
2 asx_companies_df['code'] = "ASX:" + asx_companies_df['ASX code']
3 asx_companies_df
4
5 # article_df['stock_codes'] is a cell containing a list of stock code
6 # create a new column 'stock_code_instustries' which is a list of industries for each stock code found in the asx_companies_df
  ['GICS industry group']
7 def get_industries(stock_codes):
8     if not stock_codes: # Check if the list is empty
9         return None
10    industries = asx_companies_df.loc[asx_companies_df['code'].isin(stock_codes), 'GICS industry group']
11    return set(industries) if not industries.empty else None
12
13 # Create a column for stock code industries (list of industries)
14 article_df_raw['stock_code_industries'] = article_df_raw['stock_codes'].apply(get_industries)
15
16 # Create a column for stock code industry (single industry)
17 def get_most_popular_industry(industries):
18     if industries is None or not industries: # Handle None or empty set
19         return None
20     industry_counts = pd.Series(list(industries)).value_counts()
21     return industry_counts.idxmax()
22
23 article_df_raw['stock_code_industry'] = article_df_raw['stock_code_industries'].apply(get_most_popular_industry)
24 article_df_raw
25
26 # print out a list of all unique stock code industries
27 unique_industries = set()
28 for industries in article_df_raw['stock_code_industries']:
29     if industries: # Check if industries is not None
30         unique_industries.update(industries)
31 unique_industries
32
33 industry_long_name_to_industry_short_name = {
34     "Automobiles & Components": "Auto & Parts",
35     "Banks": "Banking",
36     "Capital Goods": "Industrial",
37     "Class Pend": "Classification Pending",
38     "Commercial & Professional Services": "Commercial Services",
39     "Consumer Discretionary Distribution & Retail": "Retail",
40     "Consumer Durables & Apparel": "Consumer Durables",
41     "Consumer Services": "Consumer Services",
42     "Energy": "Energy",
43     "Equity Real Estate Investment Trusts (REITs)": "Real Estate Invstmt.",
44     "Financial Services": "Financial Services",
45     "Food, Beverage & Tobacco": "F&B & Tobacco",
46     "Health Care Equipment & Services": "Medical Equipment & Services",
47     "Household & Personal Products": "Household & Personal Goods",
48     "Materials": "Materials",
49     "Media & Entertainment": "Media & Entertainment",
50     "Not Applic": "Not Applicable",
51     "Pharmaceuticals, Biotechnology & Life Sciences": "Pharma & Biotech",
52     "Real Estate Management & Development": "Real Estate Mgt & Dev",
53     "Semiconductors & Semiconductor Equipment": "Semiconductors",
54     "Software & Services": "Software & Services",
55     "Technology Hardware & Equipment": "Tech Hardware",
56     "Telecommunication Services": "Telecom Services",
57     "Transportation": "Transportation",
58     "Utilities": "Utilities",
59 }
60 broad_sector_map = {
61     "Auto & Parts": "Consumer Discretionary",
62     "Banking": "Banking",
63     "Capital Goods": "Industrial",
64     "Classification Pending": "Other",
65     "Commercial Services": "Industrial",
66     "Retail": "Retail",
67     "Consumer Durables": "Consumer Discretionary",
68     "Consumer Services": "Consumer Discretionary",
69     "Energy": "Energy",
70     "Real Estate Invstmt.": "Real Estate",
71     "Financial Services": "Financial Services",
72     "F&B & Tobacco": "Consumer Staples",
73     "Medical Equipment & Services": "Healthcare",
74     "Household & Personal Goods": "Consumer Staples",
75     "Materials": "Materials",
76     "Media & Entertainment": "Media",
77     "Not Applicable": "Other",
78     "Pharma & Biotech": "Healthcare",
79     "Real Estate Mgt & Dev": "Real Estate",
80     "Semiconductors": "Tech",
81     "Software & Services": "Tech",
82     "Tech Hardware": "Tech",
83     "Telecom Services": "Telecom",
84     "Transportation": "Transportation",
85     "Utilities": "Utilities",
86 }
87
88 #map the long name to the short name in the dataframe
89 article_df_raw['stock_code_industry'] = article_df_raw['stock_code_industry'].map(industry_long_name_to_industry_short_name).
  map(broad_sector_map)
90 df = article_df_raw
91 df = df.dropna(subset=['document']).reset_index(drop=True)
92 df = df.loc[df['sector'] != 'Hot Topics']
93 df

```

```
94
95 # add a datetime column to the dataframe using pd.to_datetime
96 df['datetime'] = pd.to_datetime(df['date_string'], format='%B %d, %Y', errors='coerce')
97 df.to_csv('../data/article_scraped_data.csv', index=False)
```